

Section Library поверх Level A: оцінка моделі і рекомендація

30.09.2026 · аналіз без змін у репозиторії · відповідь на 20 питань власника продукту

Висновок: модель технічно правильна, і Level A відкочувати не доведеться.

Гібридна модель (Level A як основа, редакторська семантична розмітка лише для секцій бібліотеки) підходить до цього репозиторію і нічого в ньому не ламає. Одне уточнення: семантичні класи роблять редагування **безпечнішим**, але не **безпечним**. Гарантію дають ще три речі: один шаблон на компонент, контракт контенту і валідатор.

Що показав код

Level A зробив спільними CSS і JS, але **не розмітку**. Справжній спільний шаблон секції поки один — `build/banner.js`. Решта «однакових» секцій скопійована руками в генератори сторінок, і копії вже розійшлися.

- **FAQ скопійований у 12 генераторах**. Копії відрізняються відступами (`lg:py-28` проти `lg:py-32`), фоном (білий проти `ink-50`) і сіткою (три різні пропорції колонок). У Turnitin у відповіді є зайва обгортка, на головній — вступ і посилання на допомогу.
- **lifecycle-rail** під однією назвою має три різні дизайни.
- **Константи дизайну** (`h2` , `intro` , `eyebrow`) скопійовані приблизно в 10 файлів.
- **Реально перевикористовуються** близько 17 компонентів: `faq` (12 сторінок), `cta-band` (11), `banner` (7), `report-dark` (6), `lifecycle-rail` (6), `hero-checker` (5) та ще кілька на 2–3 сторінках. Решта ~60 — унікальні акти конкретних master-сторінок.
- **Два джерела правди вже існують**: `site.css` пише кольори hex-літералами замість токенів, а кнопки на сторінках зібрані з утиліт, хоча в CSS є готовий рецепт `.btn`.

Що з цього впливає. Основна робота — не «замінити класи», а звести кожен компонент бібліотеки до одного шаблону і вирішити, які з наявних розбіжностей стають варіантами, а які треба об'єднати.

Рекомендована модель

Твоя гібридна модель, доповнена трьома речами:

- 1 **Один шаблон на компонент** у `build/sections/<name>.js`. Master-сторінки теж викликають цей шаблон, копій більше немає.

2 **Контракт контенту:** які поля можна редагувати (текст, посилання, зображення, labels) і закритий список варіантів через `data-*`.

3 **Валідатор** (`check-library.js`), який перевіряє контракт навіть на HTML, вставленому в CMS.

Найнадійніший варіант безпечного редагування — **блоки з полями в CMS**, де редактор узагалі не бачить HTML. Тому до старту варто спитати розробника: що вміє наша CMS — блоки з полями чи лише сирий HTML?

Стили секцій бібліотеки

- Звичайні CSS-правила з `@apply` і `theme()`.
- Компілюються тим самим прогоном Tailwind: блок у `tailwind.src.css` між `components` і `utilities`.
- **Поза `@layer`**: `@layer components` у Tailwind v3 викидає класи, яких не бачить у файлах проекту, тож секція, що живе лише в CMS, втратила б свій CSS.
- Складні речі (маски, orbs, стани FAQ) лишаються звичайним CSS.

Як не отримати два джерела правди

- Токени — лише `tailwind.config.js`; з нього ж генеруються CSS-змінні.
- Один шаблон на компонент.
- Властивість задає **або** клас, **або** утиліта. Утиліти — лише в «запечатаних» декоративних блоках.
- Лінт перевіряє ці правила автоматично.

Чому правило «клас або утиліта» важливе саме тут: `site.css` завантажується перед `tailwind.css`, тож утиліта на тому самому елементі завжди перемагає клас. Цю пастку вже помічено на практиці: на сторінці Reviews правило приховування карток спершу програвало утиліті `flex`.

Відповіді на 20 питань

1 Чи правильно зрозуміла кінцева ціль?

Так. Точніше її сформулювати як розділення контенту і презентації: редагування контенту не може зачепити дизайн, а дизайн змінюється централізовано. Семантичні класи — засіб для цього, а не сама ціль.

2 Чи реально зробити це поверх Level A без перебудови сайту?

Так. `data-component`, JS-модулі за `data-*`, `all` id з префіксом екземпляра, стан без JS і reuse-тест не залежать від того, звідки береться стиль. Унікальні акти сторінок можна не чіпати.

3 Чи є кращий підхід?

Не заміна, а доповнення: шаблон, контракт і валідатор (див. вище), плюс спільні примітиви (шапка секції, контейнер, картка, кнопки) замість суцільного BEM. Web components і CSS modules відкинуто: вони ламають спільний CSS, SEO, роботу без JS і копіювання HTML.

4 Чим писати стилі reusable-секцій?

Комбінацією: `@apply` + звичайний CSS, в одному прогоні Tailwind, поза `@layer`. `@apply` дає рівно ті самі декларації, що й утиліти, тож parity 0 рх досяжна механічно, а адаптивні варіанти генеруються автоматично.

5 Як уникнути двох джерел правди?

Чотири правила: токени лише з конфігу; один шаблон на компонент; властивість задає або клас, або утиліта; лінт це перевіряє. Варіанти пишуться через `:where([data-...])`, щоб не піднімати специфічність.

6 Що робити семантичним, а що лишити на утилітах?

Семантичне — все, що редактор бачить або копіює: корінь секції, контейнер, шапка секції, список або сітка, картка, кнопки, зображення, пари FAQ. **Утиліти в «запечатаному» блоці** — декор: SVG-ілюстрації, діаграми, макети звітів. Класи-хуки анімацій (`.rv`, `.pen-word` тощо) лишаються.

7 Чи може редактор безпечно міняти лише текст, посилання, зображення, labels і дозволені параметри?

Гарантовано — лише через блоки з полями в CMS. Із сирим HTML — «переважно безпечно»: можна видалити обгортку або продублювати пункт FAQ з тим самим id; це ловить валідатор. Окремий ризик: підкреслення й обведення слів малюються під довжину слова, і при зміні тексту їх геометрія псується. Їх треба зробити стійкими до довжини або вимикати.

8 Чи не зламає це data-component, JS, all id, кілька екземплярів, no-JS і reuse-тести?

Ні, якщо класи, які читають JS і harness (`.faq-item`, `.faq-q`, `.faq-a`), не перейменовувати, або перейменовувати разом із модулем в одному коміті. Id пише шаблон з префіксом екземпляра, як зараз; у копіях у CMS унікальність перевіряє валідатор.

9 Чи збережемо visual parity через harness?

Так. Harness порівнює обчислені стилі (65 властивостей), розміри елементів і пікселі з порогом 0, а не назви класів. Заміна утиліт на класи з тим самим результатом проходить. Щоб прибрати зайві обгортки, harness доведеться розширити, бо зараз він порівнює послідовність елементів. Об'єднання розбіжних варіантів — дизайн-рішення, яке фіксується як прийнята відмінність після твого схвалення.

10 Наскільки великий refactor порівняно з Level A?

Середній. Нової інфраструктури майже немає; основна робота — консолідувати копії розмітки і прийняти рішення щодо варіантів.

11 Що буде основною роботою?

1. Шаблони `build/sections/*.js` і заміна копій у генераторах — найбільша частина.
 2. Перенесення утиліт у library CSS через `@apply`.
 3. Невелика зміна збірки: блок у `tailwind.src.css`, нова папка в `content`, CSS-змінні токенів.
 4. Лінт, каталог секцій, оновлення page-checkerів і README.
-

12 Які ризики?

Див. таблицю ризиків нижче.

13 Для яких секцій це робити не варто?

Для секцій, де вигляд визначають дані або код: чекер, прайсинг-віджет, макети звітів, діаграми, приклад коду, архів новин, стіна відгуків. У бібліотеці вони можуть бути «запечатаними» з 2–3 полями. Довгий текст (стаття, legal) — навпаки найпростіший: один клас-обгортка.

14 Для всіх секцій чи лише для бібліотеки?

Лише для бібліотеки, приблизно 10–15 компонентів, плюс спільна шапка секції — вона дешева і корисна всюди. Переписувати ~60 унікальних актів немає сенсу: їх ніхто не копіюватиме.

15 Де межа між library-компонентом і частиною master-сторінки?

Компонент іде в бібліотеку, якщо виконуються всі умови: потрібен на 2+ URL; контент вкладається у стабільну схему полів; верстка не підігнана під конкретний текст або кількість елементів; немає даних чи JS конкретної сторінки; ти готовий зафіксувати список варіантів. Усе інше — акт master-сторінки.

16 Чи можна буде збирати сторінки з блоків без знання Tailwind?

Так, для секцій бібліотеки. Потрібні базове розуміння HTML і довідник полів та варіантів; з блоками з полями в CMS — не потрібне й це.

17 Чи можна змінити дизайн компонента в одному місці?

Так, і це головний аргумент на користь моделі. Зараз HTML з утилітами, вставлений у CMS, доведеться правити в кожній копії; після переходу правка CSS розходить на всі сторінки. Межа: зміни структури HTML централізовано не розійдуться, тому контракт розширюється лише додаванням.

18 Що з Level A доведеться відкотити?

Нічого суттєвого. Без змін лишаються `page.js`, збірка `assets`, JS-модулі, `data-*`, `ally`, `pricing-data`, `GSAP`, `harness` і `reuse`-тест. Змінюються шаблони секцій у генераторах, `tailwind.src.css`, налаштування `content`, частина `page-checker`ів і `README`.

19 Скільки часу?

Level A зайняв близько 6 календарних днів. Бібліотека: пілот — 1–2 дні, 10–15 компонентів — ще приблизно 5–8 робочих днів. Тобто одного порядку з Level A або менше. Темп визначатимуть рішення щодо варіантів, а не код.

20 Найкраща фінальна архітектура для нашого процесу

- **Токени** — лише `tailwind.config.js`; утиліти, `library CSS` і `CSS`-змінні беруть значення звідти.
- **Assets** — `site.css`, `tailwind.css`, `site.js` деплоються на staging один раз і з версією; сторінки CMS не несуть власного `CSS/JS`.
- **Бібліотека** — один шаблон на компонент зі схемою полів і списком варіантів; master-сторінки викликають ті самі шаблони.
- **Каталог секцій** (`section-library.html`) — водночас документація, джерело для копіювання і baseline для parity.
- **CMS** — найкраще блоки з полями за схемою; якщо CMS вміє лише HTML, копіюємо з каталогу і проганяємо валідатор на staging.

Чи є фундаментальна проблема?

Ні. Є дві речі, які треба знати заздалегідь:

1. **Семантичні класи без одного шаблону на компонент нічого не консолідують.** Сьогодні «той самий» FAQ існує в 12 варіантах розмітки; справжня робота — вирішити, які розбіжності стають варіантами.
2. **Безпеку редагування гарантують поля в CMS, а не класи.** Тому першим кроком варто з'ясувати можливості CMS.

Простішої моделі, яка дає і централізовану зміну дизайну, і безпечне редагування, не видно.

Ризики

РИЗИК	РІВЕНЬ	ЯК ЗАКРИВАЄМО
Візуальні регресії	низький	parity 0 px і 65 властивостей на кожній сторінці з компонентом
Адаптивні регресії	низький	@apply з sm: / lg: і parity на 375, 768, 1440
Зростання CSS	низький	tailwind.css трохи зменшиться, library CSS додасть кілька КБ

РИЗИК	РІВЕНЬ	ЯК ЗАКРИВАЄМО
Специфічність і каскад	середній	однокласові селектори, <code>:where()</code> для варіантів, правильне місце в каскаді
Дублювання, два джерела правди	середній	один шаблон, токени, лінт «клас або утиліта»
Супровід	середній	README, каталог, контракт з версією
Сумісність із CMS і staging	середній	з'ясувати можливості CMS до пілота; валідатор на staging
Дублікати id у копіях у CMS	середній	валідатор
Підкреслення й обведення слів залежать від довжини тексту	середній	зробити стійкими до довжини або параметром
Складність варіантів	високий	фіксований список значень <code>data-*</code> , кожен варіант показаний у каталозі

Поетапний план

Нічого не виконується без окремого погодження.

ЕТАП 0

Рішення власника продукту. Список компонентів бібліотеки (приблизно 10–15), конвенція назв, варіанти для кожного компонента. Питання розробнику про можливості CMS.

ЕТАП 1

Інфраструктура, ~пів дня. Папка `build/sections/`, блок `library CSS` у збірці, CSS-змінні токенів, каркас валідатора.

ЕТАП 2

Пілот, 1–2 дні. Шапка секції і FAQ на всіх 12 сторінках: той самий скелет і класи-хуки, наявні розбіжності як варіанти `data-*`, `parity 0` рх на кожній сторінці. Потім одна секція із зображенням і посиланнями. Власник продукту дивиться на HTML і вирішує, чи він достатньо «редакторський».

ЕТАП 3

Спрощення скелета (за бажанням). Прибрати зайві обгортки, розширивши `harness`.

ЕТАП 4

Розгортання за частотою: `cta-band` → `banner` → `lifecycle-rail` → `inquiry-form` → ...

ЕТАП 5

Каталог і документація для розробника staging: поля, варіанти, правила.